

FIT – HPUni BECOME A BETTER DEVELOPER

=====

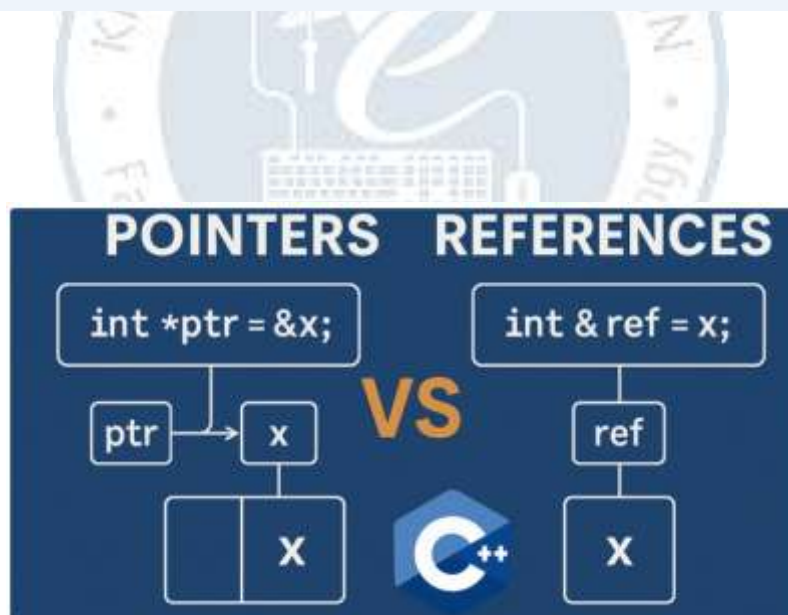
BỘ BÀI TẬP THỰC HÀNH CON TRỎ TRONG C++

30 bài: con trỏ cơ bản - con trỏ và mảng - mảng con trỏ - các loại con trỏ khác

Mỗi bài gồm: Nêu bài toán - Input - Output - Ví dụ - Thuật toán - Code minh họa

15 bài có sử dụng hàm tự định nghĩa (ký hiệu ★) | Chuẩn C++11

Mục tiêu: rèn luyện cách lưu địa chỉ, giải tham chiếu, thao tác con trỏ với mảng, con trỏ nhiều cấp, con trỏ hàm, con trỏ tới thành viên và smart pointer. Tài liệu ưu tiên bài toán ngắn, có thể chạy độc lập để dùng trong giờ thực hành hoặc giao bài tự luyện.



HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

Tài liệu được xây dựng theo mẫu thống nhất cho cả 30 bài: (1) Nêu bài toán, (2) Input, (3) Output, (4) Ví dụ, (5) Thuật toán giải và (6) Code minh họa.

Ký hiệu ★ đặt sau tên bài cho biết bài đó có sử dụng hàm tự định nghĩa. Tổng cộng có đúng 15 bài dùng hàm, phân bố từ bài cơ bản đến bài về con trỏ hàm và con trỏ tới thành viên.

Các chương trình được viết theo chuẩn C++11 và đã được kiểm tra biên dịch, đồng thời chạy thử với dữ liệu ví dụ trong từng bài.

Cơ sở tham khảo từ tài liệu đính kèm

- Bjarne Stroustrup, The C++ Programming Language, Fourth Edition: Chương 7 về pointers, arrays và quan hệ giữa tên mảng với con trỏ phần tử đầu.
- Phần nullptr được dùng để biểu diễn “không có đối tượng”; trước khi giải tham chiếu cần bảo đảm con trỏ hợp lệ.
- Số học con trỏ chỉ được sử dụng có kiểm soát trong phạm vi cùng một mảng; con trỏ one-past-the-end có thể dùng làm mốc kết thúc nhưng không được giải tham chiếu.
- Chương 11 về new/delete và quản lý vùng nhớ động; bài tập nhấn mạnh ghép new với delete và new[] với delete[].
- Chương 12 và Chương 20 cung cấp nền tảng cho con trỏ hàm và con trỏ tới thành viên; Chương 34 liên quan đến smart pointer.

Quy ước và lưu ý an toàn

- Không giải tham chiếu nullptr, con trỏ one-past-the-end hoặc con trỏ đã trỏ tới vùng nhớ bị giải phóng.
- Không trừ hai con trỏ nếu chúng không cùng thuộc một mảng (hoặc vị trí one-past của mảng đó).
- Mảng truyền vào hàm kiểu T[] được xem như T*; kích thước mảng không tự đi kèm, vì vậy bài tập truyền thêm n hoặc dùng cặp con trỏ first/last.
- void* chỉ nên dùng trong tình huống mức thấp; phải ép về đúng kiểu trước khi truy cập đối tượng.
- Trong code thực tế hiện đại, ưu tiên RAII và smart pointer cho quyền sở hữu tài nguyên; raw pointer trong bộ bài này chủ yếu dùng để học cơ chế ngôn ngữ.

Phân bố 30 bài tập

Phần	Bài	Số bài	Trọng tâm
I	1-8	8	Con trỏ cơ bản, nullptr, const, new/delete
II	9-19	11	Con trỏ với mảng, pointer arithmetic, mảng động, pointer-to-array
III	20-24	5	Mảng con trỏ, con trỏ cấp hai, int**, con trỏ tới struct
IV	25-30	6	void*, con trỏ hàm, pointer-to-member, smart pointer

Danh mục bài tập

Bài	Tên bài	Nhóm	Hàm
1	Truy cập và thay đổi giá trị qua con trỏ	Con trỏ cơ bản	
2	Hoán đổi hai số bằng con trỏ	Con trỏ cơ bản	★
3	Tìm số lớn nhất trong ba số bằng con trỏ	Con trỏ cơ bản	
4	Trả về thương và số dư qua hai con trỏ	Con trỏ cơ bản	★
5	Cấp phát động một số nguyên	Con trỏ cơ bản	
6	Kiểm tra nullptr trước khi giải tham chiếu	Con trỏ cơ bản	
7	Con trỏ tới hằng: tìm giá trị nhỏ hơn	Con trỏ cơ bản	
8	Con trỏ hằng: cập nhật đối tượng tại địa chỉ cố định	Con trỏ cơ bản	

9	Tính tổng mảng bằng phép duyệt con trỏ	Con trỏ và mảng	★
10	Tìm phần tử lớn nhất bằng con trỏ	Con trỏ và mảng	★
11	Đếm số dương bằng con trỏ	Con trỏ và mảng	★
12	Đảo ngược mảng bằng hai con trỏ	Con trỏ và mảng	★
13	Tìm lần xuất hiện đầu tiên và trả về con trỏ	Con trỏ và mảng	★
14	Sắp xếp tăng dần bằng số học con trỏ	Con trỏ và mảng	★
15	Sao chép mảng bằng hai con trỏ	Con trỏ và mảng	
16	Tích vô hướng của hai mảng bằng con trỏ	Con trỏ và mảng	
17	Khoảng cách giữa hai vị trí bằng phép trừ con trỏ	Con trỏ và mảng	
18	Mảng động: tính trung bình và đếm phần tử trên trung bình	Con trỏ và mảng	
19	Con trỏ tới mảng: tính tổng từng hàng	Con trỏ và mảng	
20	Sắp xếp mảng con trỏ mà không đổi mảng gốc	Mảng con trỏ và con trỏ nhiều cấp	★
21	Mảng con trỏ chuỗi: tìm từ dài nhất	Mảng con trỏ và con trỏ nhiều cấp	★
22	Con trỏ cấp hai: cấp phát biến trong hàm	Mảng con trỏ và con trỏ nhiều cấp	★
23	Ma trận động bằng int** và tính tổng	Mảng con trỏ và con trỏ nhiều cấp	★
24	Mảng con trỏ tới cấu trúc: xếp sinh viên theo điểm	Mảng con trỏ và con trỏ nhiều cấp	
25	Con trỏ void*: lưu địa chỉ chưa biết kiểu cụ thể	Các loại con trỏ khác	
26	Con trỏ hàm: máy tính bốn phép toán	Các loại con trỏ khác	★
27	Mảng con trỏ hàm: chọn phép biến đổi	Các loại con trỏ khác	★
28	Con trỏ tới thành viên lớp	Các loại con trỏ khác	★
29	Smart pointer unique_ptr cho mảng động	Các loại con trỏ khác	
30	shared_ptr và weak_ptr: quan sát vòng đời đối tượng	Các loại con trỏ khác	

PHẦN I. CON TRỎ CƠ BẢN

Luyện địa chỉ, giải tham chiếu, nullptr, const pointer/pointer-to-const và cấp phát động một đối tượng.

Bài 1. Truy cập và thay đổi giá trị qua con trỏ

1. Nêu bài toán

Nhập một số nguyên x. Tạo con trỏ trỏ tới x, dùng phép giải tham chiếu để nhân giá trị của x lên 2 và in kết quả.

2. Input

Một số nguyên x ($-10^6 \leq x \leq 10^6$).

3. Output

Giá trị của x sau khi được nhân 2 thông qua con trỏ.

4. Ví dụ

Input	Output
12	24

5. Thuật toán giải

Bước 1: Đọc x.

Bước 2: Khai báo `int* p = &x` để p lưu địa chỉ của x.

Bước 3: Thực hiện `*p = *p * 2`.

Bước 4: In x; giá trị của x đã thay đổi vì *p là chính đối tượng x.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cin >> x;

    int* p = &x;
    *p = *p * 2;

    cout << x;
    return 0;
}
```

Bài 2. Hoán đổi hai số bằng con trỏ ★

1. Nêu bài toán

Nhập hai số nguyên a và b. Viết hàm nhận địa chỉ của hai biến và hoán đổi giá trị của chúng.

2. Input

Hai số nguyên a, b.

3. Output

Hai giá trị sau khi hoán đổi, cách nhau bởi một khoảng trắng.

4. Ví dụ

Input	Output
8 15	15 8

5. Thuật toán giải

Bước 1: Viết hàm swapValues(int* a, int* b).

Bước 2: Dùng biến tạm để đổi *a và *b.

Bước 3: Trong main, truyền &a và &b cho hàm.

Bước 4: In a và b sau khi hàm kết thúc.

6. Code minh họa

```
#include <iostream>
using namespace std;

void swapValues(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int a, b;
    cin >> a >> b;
    swapValues(&a, &b);
    cout << a << ' ' << b;
    return 0;
}
```

Bài 3. Tìm số lớn nhất trong ba số bằng con trỏ**1. Nêu bài toán**

Nhập ba số nguyên a, b, c. Sử dụng các con trỏ để xác định biến đang chứa giá trị lớn nhất và in giá trị đó.

2. Input

Ba số nguyên a, b, c.

3. Output

Giá trị lớn nhất trong ba số.

4. Ví dụ

Input	Output
7 12 5	12

5. Thuật toán giải

Bước 1: Tạo pa, pb, pc lần lượt trỏ tới a, b, c.

Bước 2: Cho pMax ban đầu trỏ tới a.

Bước 3: So sánh *pb, *pc với *pMax và cập nhật pMax khi gặp giá trị lớn hơn.

Bước 4: In *pMax.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
```

```

int* pa = &a;
int* pb = &b;
int* pc = &c;
int* pMax = pa;

if (*pb > *pMax) pMax = pb;
if (*pc > *pMax) pMax = pc;

cout << *pMax;
return 0;
}

```

Bài 4. Trả về thương và số dư qua hai con trỏ ★

1. Nêu bài toán

Nhập hai số nguyên a và b (b khác 0). Viết hàm tính đồng thời thương nguyên và số dư, trả hai kết quả qua hai tham số con trỏ.

2. Input

Hai số nguyên a, b; b != 0.

3. Output

Thương nguyên và số dư của a chia b.

4. Ví dụ

Input	Output
17 5	3 2

5. Thuật toán giải

Bước 1: Viết hàm divMod(a, b, q, r), trong đó q và r là con trỏ.

Bước 2: Gán *q = a / b và *r = a % b.

Bước 3: Trong main, khai báo q, r rồi truyền &q, &r cho hàm.

Bước 4: In q và r.

6. Code minh họa

```

#include <iostream>
using namespace std;

void divMod(int a, int b, int* q, int* r) {
    *q = a / b;
    *r = a % b;
}

int main() {
    int a, b, q, r;
    cin >> a >> b;
    divMod(a, b, &q, &r);
    cout << q << ' ' << r;
    return 0;
}

```

Bài 5. Cấp phát động một số nguyên

1. Nêu bài toán

Nhập số nguyên x. Cấp phát động một biến int trên free store, lưu x vào vùng nhớ đó, tính bình phương và giải phóng bộ nhớ.

2. Input

Một số nguyên x với $|x| \leq 30000$.

3. Output

Bình phương của x.

4. Ví dụ

Input	Output
6	36

5. Thuật toán giải

Bước 1: Đọc x.

Bước 2: Cấp phát int bằng `new int{x}` và nhận địa chỉ vào p.

Bước 3: Tính `(*p) * (*p)` và in kết quả.

Bước 4: Gọi `delete p` đúng một lần sau lần sử dụng cuối.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cin >> x;

    int* p = new int{x};
    cout << (*p) * (*p);

    delete p;
    return 0;
}
```

Bài 6. Kiểm tra nullptr trước khi giải tham chiếu

1. Nêu bài toán

Nhập một điểm thực s. Nếu s thuộc đoạn $[0, 10]$, cho con trỏ trỏ tới s và in lại điểm; nếu không, dùng nullptr để biểu diễn dữ liệu không hợp lệ và in INVALID.

2. Input

Một số thực s.

3. Output

Nếu $0 \leq s \leq 10$, in s với 1 chữ số thập phân; ngược lại in INVALID.

4. Ví dụ

Input	Output
11	INVALID

5. Thuật toán giải

Bước 1: Đọc s.

Bước 2: Nếu s hợp lệ, đặt $p = \&s$; ngược lại đặt $p = \text{nullptr}$.

Bước 3: Chỉ giải tham chiếu khi p khác nullptr .

Bước 4: In INVALID nếu p là nullptr .

6. Code minh họa

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double s;
    cin >> s;

    double* p = (0.0 <= s && s <= 10.0) ? &s : nullptr;
    if (p == nullptr)
        cout << "INVALID";
    else
        cout << fixed << setprecision(1) << *p;

    return 0;
}
```

Bài 7. Con trỏ tới hằng: tìm giá trị nhỏ hơn

1. Nêu bài toán

Nhập hai số nguyên a, b . Dùng con trỏ tới hằng (pointer-to-const) để trỏ tới số nhỏ hơn và in giá trị đó.

2. Input

Hai số nguyên a, b .

3. Output

Số nhỏ hơn trong hai số.

4. Ví dụ

Input	Output
14 9	9

5. Thuật toán giải

Bước 1: Đọc a, b .

Bước 2: Khai báo $\text{const int}^* p$.

Bước 3: Cho p trỏ tới a nếu $a \leq b$, ngược lại trỏ tới b .

Bước 4: In $*p$. Con trỏ có thể đổi nơi trỏ nhưng không sửa giá trị qua p .

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;

    const int* p;
    if (a <= b) p = &a;
    else p = &b;
}
```

```
cout << *p;
return 0;
}
```

Bài 8. Con trỏ hằng: cập nhật đối tượng tại địa chỉ cố định

1. Nêu bài toán

Nhập x và k. Tạo một con trỏ hằng luôn trỏ tới x, sau đó tăng x thêm k thông qua con trỏ và in kết quả.

2. Input

Hai số nguyên x, k.

3. Output

Giá trị x + k.

4. Ví dụ

Input	Output
20 -3	17

5. Thuật toán giải

Bước 1: Đọc x, k.

Bước 2: Khai báo `int* const p = &x`.

Bước 3: Dùng `*p += k` để thay đổi đối tượng x.

Bước 4: In x. Bản thân p không thể được gán sang địa chỉ khác.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int x, k;
    cin >> x >> k;

    int* const p = &x;
    *p += k;

    cout << x;
    return 0;
}
```

PHẦN II. CON TRỎ VÀ MẢNG

Luyện quan hệ mảng - con trỏ, duyệt [first,last), số học con trỏ, mảng động và con trỏ tới mảng.

Bài 9. Tính tổng mảng bằng phép duyệt con trỏ ★

1. Nêu bài toán

Cho mảng n số nguyên. Viết hàm nhận con trỏ đầu và con trỏ sau phần tử cuối để tính tổng mảng.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

Tổng các phần tử.

4. Ví dụ

Input	Output
5 1 2 3 4 5	15

5. Thuật toán giải

Bước 1: Hàm sumArray(first, last) nhận đoạn [first, last).

Bước 2: Khởi tạo tổng bằng 0.

Bước 3: Trong khi first != last, cộng *first rồi tăng first.

Bước 4: Gọi hàm với a và a + n.

6. Code minh họa

```
#include <iostream>
using namespace std;

long long sumArray(const int* first, const int* last) {
    long long sum = 0;
    while (first != last) {
        sum += *first;
        ++first;
    }
    return sum;
}

int main() {
    int n, a[1000];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cout << sumArray(a, a + n);
    return 0;
}
```

Bài 10. Tìm phần tử lớn nhất bằng con trỏ ★**1. Nêu bài toán**

Cho mảng n số nguyên. Viết hàm dùng con trỏ để tìm giá trị lớn nhất.

2. Input

n và n số nguyên, $1 \leq n \leq 1000$.

3. Output

Giá trị lớn nhất của mảng.

4. Ví dụ

Input	Output
6 -4 8 2 15 7 1	15

5. Thuật toán giải

Bước 1: Cho pMax trỏ tới phần tử đầu.

Bước 2: Duyệt p từ a+1 đến a+n.

Bước 3: Nếu *p > *pMax thì gán pMax = p.

Bước 4: Trả về *pMax.

6. Code minh họa

```
#include <iostream>
using namespace std;

int maxArray(const int* a, int n) {
    const int* pMax = a;
    for (const int* p = a + 1; p != a + n; ++p)
        if (*p > *pMax) pMax = p;
    return *pMax;
}

int main() {
    int n, a[1000];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cout << maxArray(a, n);
    return 0;
}
```

Bài 11. Đếm số dương bằng con trỏ ★**1. Nêu bài toán**

Cho mảng n số nguyên. Viết hàm đếm bao nhiêu phần tử dương bằng cách duyệt con trỏ.

2. Input

n và n số nguyên.

3. Output

Số lượng phần tử > 0.

4. Ví dụ

Input	Output
7 -1 3 0 4 -5 6 0	3

5. Thuật toán giải

Bước 1: Hàm nhận a và n.

Bước 2: Duyệt p từ a đến a+n.

Bước 3: Mỗi khi *p > 0, tăng biến đếm.

Bước 4: Trả về số đếm.

6. Code minh họa

```
#include <iostream>
using namespace std;

int countPositive(const int* a, int n) {
    int count = 0;
    for (const int* p = a; p != a + n; ++p)
        if (*p > 0) ++count;
    return count;
}

int main() {
    int n, a[1000];
    cin >> n;
```

```

for (int i = 0; i < n; ++i) cin >> a[i];
cout << countPositive(a, n);
return 0;
}

```

Bài 12. Đảo ngược mảng bằng hai con trỏ ★

1. Nêu bài toán

Cho mảng n số nguyên. Viết hàm đảo ngược mảng tại chỗ bằng một con trỏ đầu và một con trỏ cuối.

2. Input

n và n số nguyên.

3. Output

Mảng sau khi đảo ngược.

4. Ví dụ

Input	Output
5 10 20 30 40 50	50 40 30 20 10

5. Thuật toán giải

Bước 1: left trỏ phần tử đầu, right trỏ phần tử cuối.

Bước 2: Trong khi $left < right$, hoán đổi $*left$ và $*right$.

Bước 3: Tăng left, giảm right.

Bước 4: In mảng sau khi hàm hoàn tất.

6. Code minh họa

```

#include <iostream>
using namespace std;

void reverseArray(int* a, int n) {
    int* left = a;
    int* right = a + n - 1;
    while (left < right) {
        int temp = *left;
        *left = *right;
        *right = temp;
        ++left;
        --right;
    }
}

int main() {
    int n, a[1000];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    reverseArray(a, n);
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}

```

Bài 13. Tìm lần xuất hiện đầu tiên và trả về con trỏ ★**1. Nêu bài toán**

Cho mảng n số nguyên và giá trị x . Viết hàm trả về con trỏ tới lần xuất hiện đầu tiên của x ; nếu không có, trả nullptr.

2. Input

Dòng 1: n . Dòng 2: n số nguyên. Dòng 3: x .

3. Output

Vị trí 1-based của x đầu tiên; nếu không có, in -1.

4. Ví dụ

Input	Output
6 4 7 2 7 9 1 7	2

5. Thuật toán giải

Bước 1: Duyệt p từ a tới $a+n$.

Bước 2: Nếu $*p == x$, trả p ngay.

Bước 3: Nếu duyệt hết, trả nullptr.

Bước 4: Trong main, nếu p khác nullptr thì vị trí là $p - a + 1$.

6. Code minh họa

```
#include <iostream>
using namespace std;

int* findFirst(int* a, int n, int x) {
    for (int* p = a; p != a + n; ++p)
        if (*p == x) return p;
    return nullptr;
}

int main() {
    int n, a[1000], x;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> x;

    int* p = findFirst(a, n, x);
    if (p == nullptr) cout << -1;
    else cout << (p - a + 1);
    return 0;
}
```

Bài 14. Sắp xếp tăng dần bằng số học con trỏ ★**1. Nêu bài toán**

Cho mảng n số nguyên. Viết hàm sắp xếp tăng dần; việc truy cập phần tử trong phần xử lý chính dùng $*(a + i)$ thay cho $a[i]$.

2. Input

n ($1 \leq n \leq 200$) và n số nguyên.

3. Output

Mảng tăng dần.

4. Ví dụ

Input	Output
6 5 1 4 2 8 3	1 2 3 4 5 8

5. Thuật toán giải

Bước 1: Dùng hai vòng lặp của Bubble Sort.

Bước 2: So sánh $*(a+j)$ và $*(a+j+1)$.

Bước 3: Nếu sai thứ tự thì hoán đổi hai giá trị qua con trỏ.

Bước 4: In mảng đã sắp xếp.

6. Code minh họa

```
#include <iostream>
using namespace std;

void bubbleSort(int* a, int n) {
    for (int i = 0; i < n - 1; ++i)
        for (int j = 0; j < n - 1 - i; ++j)
            if (*(a + j) > *(a + j + 1)) {
                int temp = *(a + j);
                *(a + j) = *(a + j + 1);
                *(a + j + 1) = temp;
            }
}

int main() {
    int n, a[200];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    bubbleSort(a, n);
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}
```

Bài 15. Sao chép mảng bằng hai con trỏ**1. Nêu bài toán**

Cho mảng a gồm n số nguyên. Sao chép a sang b chỉ bằng hai con trỏ nguồn và đích, sau đó in b.

2. Input

n và n số nguyên.

3. Output

Mảng b sau khi sao chép.

4. Ví dụ

Input	Output
-------	--------

4
9 2 6 1

9 2 6 1

5. Thuật toán giải

Bước 1: Cho $src = a$ và $dest = b$.

Bước 2: Lặp n lần: gán $*dest = *src$.

Bước 3: Tăng đồng thời src và $dest$.

Bước 4: In mảng b .

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int n, a[1000], b[1000];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];

    const int* src = a;
    int* dest = b;
    for (int i = 0; i < n; ++i) {
        *dest = *src;
        ++src;
        ++dest;
    }

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << b[i];
    }
    return 0;
}
```

Bài 16. Tích vô hướng của hai mảng bằng con trỏ

1. Nêu bài toán

Cho hai mảng số nguyên a và b cùng n phần tử. Tính tích vô hướng $a[0]*b[0] + \dots + a[n-1]*b[n-1]$ bằng con trỏ.

2. Input

Dòng 1: n . Dòng 2: mảng a . Dòng 3: mảng b .

3. Output

Giá trị tích vô hướng.

4. Ví dụ

Input	Output
3 1 2 3 4 5 6	32

5. Thuật toán giải

Bước 1: Cho $pa = a$, $pb = b$.

Bước 2: Lặp n lần, cộng $(*pa) * (*pb)$ vào tổng.

Bước 3: Mỗi vòng tăng pa và pb.

Bước 4: In tổng.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int n, a[1000], b[1000];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < n; ++i) cin >> b[i];

    const int* pa = a;
    const int* pb = b;
    long long dot = 0;
    for (int i = 0; i < n; ++i, ++pa, ++pb)
        dot += 1LL * (*pa) * (*pb);

    cout << dot;
    return 0;
}
```

Bài 17. Khoảng cách giữa hai vị trí bằng phép trừ con trỏ

1. Nêu bài toán

Cho mảng n phần tử và hai chỉ số i, j hợp lệ (0-based). Dùng phép trừ hai con trỏ để tính số phần tử chênh lệch từ vị trí i tới j.

2. Input

Dòng 1: n. Dòng 2: n số nguyên. Dòng 3: i j với $0 \leq i \leq j < n$.

3. Output

Giá trị $(&a[j] - &a[i])$, tức j - i.

4. Ví dụ

Input	Output
5 10 20 30 40 50 1 4	3

5. Thuật toán giải

Bước 1: Tạo $pi = &a[i]$, $pj = &a[j]$.

Bước 2: Vì hai con trỏ cùng nằm trong một mảng, phép $pj - pi$ là hợp lệ.

Bước 3: In $pj - pi$.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int n, a[1000], i, j;
    cin >> n;
    for (int k = 0; k < n; ++k) cin >> a[k];
    cin >> i >> j;
```

```

int* pi = &a[i];
int* pj = &a[j];
cout << (pj - pi);
return 0;
}

```

Bài 18. Mảng động: tính trung bình và đếm phần tử trên trung bình

1. Nêu bài toán

Nhập n tại thời gian chạy, cấp phát mảng động n số thực, tính trung bình và đếm số phần tử lớn hơn trung bình.

2. Input

Dòng 1: n ($1 \leq n \leq 10000$). Dòng 2: n số thực.

3. Output

Dòng 1: trung bình với 2 chữ số thập phân. Dòng 2: số phần tử lớn hơn trung bình.

4. Ví dụ

Input	Output
5 1 2 3 4 5	3.00 2

5. Thuật toán giải

Bước 1: Cấp phát `double* a = new double[n]`.

Bước 2: Nhập dữ liệu và tính tổng.

Bước 3: Tính `mean = sum / n`, duyệt lại để đếm `a[i] > mean`.

Bước 4: In kết quả rồi `delete[] a`.

6. Code minh họa

```

#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cin >> n;
    double* a = new double[n];

    double sum = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        sum += a[i];
    }

    double mean = sum / n;
    int count = 0;
    for (double* p = a; p != a + n; ++p)
        if (*p > mean) ++count;

    cout << fixed << setprecision(2) << mean << '\n' << count;
    delete[] a;
    return 0;
}

```

Bài 19. Con trỏ tới mảng: tính tổng từng hàng

1. Nêu bài toán

Cho ma trận $n \times 3$ số nguyên. Dùng con trỏ tới mảng 3 phần tử để duyệt và tính tổng của từng hàng.

2. Input

Dòng 1: n ($1 \leq n \leq 100$). Tiếp theo n dòng, mỗi dòng 3 số nguyên.

3. Output

n tổng hàng trên một dòng, cách nhau bởi khoảng trắng.

4. Ví dụ

Input	Output
2 1 2 3 4 5 6	6 15

5. Thuật toán giải

Bước 1: Khai báo `int a[100][3]` và nhập dữ liệu.

Bước 2: Khai báo `int (*p)[3] = a`.

Bước 3: Với mỗi hàng i , truy cập phần tử qua `*(p+i+j)`.

Bước 4: Cộng 3 phần tử và in tổng hàng.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    int n, a[100][3];
    cin >> n;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < 3; ++j)
            cin >> a[i][j];

    int (*p)[3] = a;
    for (int i = 0; i < n; ++i) {
        int sum = 0;
        for (int j = 0; j < 3; ++j)
            sum += (*(p + i) + j);
        if (i) cout << ' ';
        cout << sum;
    }
    return 0;
}
```

PHẦN III. MẢNG CON TRỎ VÀ CON TRỎ NHIỀU CẤP

Luyện mảng các địa chỉ, chuỗi C, `int**`, ma trận động và mảng con trỏ tới cấu trúc.

Bài 20. Sắp xếp mảng con trỏ mà không đổi mảng gốc ★

1. Nêu bài toán

Cho mảng a gồm n số nguyên. Tạo mảng con trỏ p sao cho $p[i]$ trỏ tới $a[i]$, rồi sắp xếp p theo giá trị được trỏ tới. Mảng a ban đầu không được thay đổi.

2. Input

n ($1 \leq n \leq 200$) và n số nguyên.

3. Output

Dòng 1: các giá trị theo thứ tự tăng qua mảng p . Dòng 2: mảng a gốc.

4. Ví dụ

Input	Output
5 4 1 3 2 5	1 2 3 4 5 4 1 3 2 5

5. Thuật toán giải

Bước 1: Khởi tạo $p[i] = \&a[i]$.

Bước 2: Hàm `sortPointers()` chỉ hoán đổi các địa chỉ $p[i]$, không đổi $*p[i]$.

Bước 3: So sánh $*p[j]$ và $*p[j+1]$ để sắp xếp.

Bước 4: In $*p[i]$, sau đó in $a[i]$ để thấy dữ liệu gốc giữ nguyên.

6. Code minh họa

```
#include <iostream>
using namespace std;

void sortPointers(int* p[], int n) {
    for (int i = 0; i < n - 1; ++i)
        for (int j = 0; j < n - 1 - i; ++j)
            if (*p[j] > *p[j + 1]) {
                int* temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
}

int main() {
    int n, a[200];
    int* p[200];
    cin >> n;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        p[i] = &a[i];
    }

    sortPointers(p, n);
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << *p[i];
    }
    cout << '\n';
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}
```

Bài 21. Mảng con trở chuỗi: tìm từ dài nhất ★**1. Nêu bài toán**

Nhập n từ không chứa khoảng trắng. Lưu các từ trong mảng hai chiều char, đồng thời tạo mảng con trở tới từng từ. Viết hàm tìm từ dài nhất qua mảng con trở.

2. Input

Dòng 1: n ($1 \leq n \leq 20$). Dòng 2: n từ, mỗi từ dài tối đa 100 ký tự.

3. Output

Từ dài nhất; nếu hòa, lấy từ xuất hiện trước.

4. Ví dụ

Input	Output
4 cat elephant dog tiger	elephant

5. Thuật toán giải

Bước 1: Mỗi $p[i]$ trở tới đầu chuỗi $words[i]$.

Bước 2: Hàm `longest()` giữ con trở best tới chuỗi dài nhất hiện tại.

Bước 3: So sánh `strlen(p[i])` với `strlen(best)`.

Bước 4: Trả về best và in chuỗi được trở tới.

6. Code minh họa

```
#include <iostream>
#include <cstring>
using namespace std;

char* longest(char* p[], int n) {
    char* best = p[0];
    for (int i = 1; i < n; ++i)
        if (strlen(p[i]) > strlen(best)) best = p[i];
    return best;
}

int main() {
    int n;
    char words[20][101];
    char* p[20];
    cin >> n;
    for (int i = 0; i < n; ++i) {
        cin >> words[i];
        p[i] = words[i];
    }
    cout << longest(p, n);
    return 0;
}
```

Bài 22. Con trở cấp hai: cấp phát biến trong hàm ★**1. Nêu bài toán**

Nhập số nguyên x. Viết hàm nhận `int**` để cấp phát một biến `int` động, gán x cho biến đó. Sau khi hàm trả về, in giá trị và bình phương rồi giải phóng bộ nhớ.

2. Input

Một số nguyên x với $|x| \leq 30000$.

3. Output

x và x^2 , cách nhau bởi khoảng trắng.

4. Ví dụ

Input	Output
7	7 49

5. Thuật toán giải

Bước 1: Trong main, khởi tạo `int* p = nullptr`.

Bước 2: Hàm `createNumber(int** p, x)` thực hiện `*p = new int{x}`.

Bước 3: Gọi `createNumber(&p, x)`, sau đó dùng `*p`.

Bước 4: `delete p` sau khi sử dụng cuối.

6. Code minh họa

```
#include <iostream>
using namespace std;

void createNumber(int** p, int value) {
    *p = new int{value};
}

int main() {
    int x;
    cin >> x;

    int* p = nullptr;
    createNumber(&p, x);
    cout << *p << ' ' << (*p) * (*p);

    delete p;
    return 0;
}
```

Bài 23. Ma trận động bằng `int**` và tính tổng ★**1. Nêu bài toán**

Nhập số hàng r và số cột c, cấp phát ma trận động bằng `int**`, nhập các phần tử và viết hàm tính tổng toàn ma trận.

2. Input

Dòng 1: r c ($1 \leq r, c \leq 100$). Sau đó r*c số nguyên.

3. Output

Tổng tất cả phần tử.

4. Ví dụ

Input	Output
2 3 1 2 3 4 5 6	21

5. Thuật toán giải

Bước 1: Cấp phát `a = new int*[r]`.

Bước 2: Với mỗi hàng i, cấp phát `a[i] = new int[c]`.

Bước 3: Nhập dữ liệu; hàm matrixSum() duyệt a[i][j] để cộng tổng.

Bước 4: Giải phóng từng delete[] a[i], cuối cùng delete[] a.

6. Code minh họa

```
#include <iostream>
using namespace std;

long long matrixSum(int** a, int r, int c) {
    long long sum = 0;
    for (int i = 0; i < r; ++i)
        for (int j = 0; j < c; ++j)
            sum += a[i][j];
    return sum;
}

int main() {
    int r, c;
    cin >> r >> c;

    int** a = new int*[r];
    for (int i = 0; i < r; ++i)
        a[i] = new int[c];

    for (int i = 0; i < r; ++i)
        for (int j = 0; j < c; ++j)
            cin >> a[i][j];

    cout << matrixSum(a, r, c);

    for (int i = 0; i < r; ++i)
        delete[] a[i];
    delete[] a;
    return 0;
}
```

Bài 24. Mảng con trỏ tới cấu trúc: xếp sinh viên theo điểm

1. Nêu bài toán

Nhập danh sách sinh viên gồm tên một từ và điểm. Tạo mảng con trỏ tới các phần tử Student, sắp xếp mảng con trỏ theo điểm giảm dần mà không đổi thứ tự mảng Student gốc.

2. Input

Dòng 1: n (1 ≤ n ≤ 50). n dòng tiếp theo: name score.

3. Output

Danh sách theo điểm giảm dần, mỗi dòng gồm tên và điểm với 1 chữ số thập phân.

4. Ví dụ

Input	Output
3	Binh 9.2
An 8.5	An 8.5
Binh 9.2	Chi 7.8
Chi 7.8	

5. Thuật toán giải

Bước 1: Nhập mảng Student a và đặt p[i] = &a[i].

Bước 2: Sắp xếp p bằng cách so sánh p[i]->score.

Bước 3: Chỉ hoán đổi các con trỏ, không hoán đổi Student.

Bước 4: In p[i]->name và p[i]->score.

6. Code minh họa

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Student {
    string name;
    double score;
};

int main() {
    int n;
    Student a[50];
    Student* p[50];
    cin >> n;
    for (int i = 0; i < n; ++i) {
        cin >> a[i].name >> a[i].score;
        p[i] = &a[i];
    }

    for (int i = 0; i < n - 1; ++i)
        for (int j = i + 1; j < n; ++j)
            if (p[i]->score < p[j]->score) {
                Student* temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }

    cout << fixed << setprecision(1);
    for (int i = 0; i < n; ++i)
        cout << p[i]->name << ' ' << p[i]->score << (i + 1 == n ?
'\n' : '\n');
    return 0;
}
```

PHẦN IV. CÁC LOẠI CON TRỎ KHÁC

Luyện void*, con trỏ hàm, mảng con trỏ hàm, con trỏ tới thành viên và smart pointer trong C++11.

Bài 25. Con trỏ void*: lưu địa chỉ chưa biết kiểu cụ thể

1. Nêu bài toán

Nhập mã kiểu t. Nếu t=1, đọc một số nguyên; nếu t=2, đọc một số thực. Dùng void* để giữ địa chỉ dữ liệu rồi chuyển lại đúng kiểu và in giá trị gấp đôi.

2. Input

t thuộc {1,2}, sau đó là một giá trị tương ứng.

3. Output

Giá trị gấp đôi. Với t=2, in 2 chữ số thập phân.

4. Ví dụ

Input	Output
17	14

5. Thuật toán giải

Bước 1: void* chỉ giữ địa chỉ, không được giải tham chiếu trực tiếp.

Bước 2: Nếu t=1, cho p trở tới int x và static_cast<int*>(p) khi dùng.

Bước 3: Nếu t=2, cho p trở tới double x và static_cast<double*>(p) khi dùng.

Bước 4: In giá trị gấp đôi sau khi ép về đúng kiểu.

6. Code minh họa

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int t;
    cin >> t;
    void* p = nullptr;

    if (t == 1) {
        int x;
        cin >> x;
        p = &x;
        cout << 2 * (*static_cast<int*>(p));
    } else {
        double x;
        cin >> x;
        p = &x;
        cout << fixed << setprecision(2)
              << 2.0 * (*static_cast<double*>(p));
    }
    return 0;
}
```

Bài 26. Con trỏ hàm: máy tính bốn phép toán ★**1. Nêu bài toán**

Nhập hai số thực a, b và toán tử +, -, *, /. Dùng một con trỏ hàm để lựa chọn phép toán tương ứng và tính kết quả.

2. Input

a b op; nếu op là / thì b khác 0.

3. Output

Kết quả với 2 chữ số thập phân.

4. Ví dụ

Input	Output
10 4 /	2.50

5. Thuật toán giải

Bước 1: Viết bốn hàm add, sub, mul, divide.

Bước 2: Khai báo kiểu con trỏ hàm double (*f)(double,double).

Bước 3: Dựa vào op, gán f tới hàm tương ứng.

Bước 4: Gọi f(a,b) và in kết quả.

6. Code minh họa

```
#include <iostream>
#include <iomanip>
using namespace std;

double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double divide(double a, double b) { return a / b; }

int main() {
    double a, b;
    char op;
    cin >> a >> b >> op;

    double (*f)(double, double) = nullptr;
    if (op == '+') f = add;
    else if (op == '-') f = sub;
    else if (op == '*') f = mul;
    else f = divide;

    cout << fixed << setprecision(2) << f(a, b);
    return 0;
}
```

Bài 27. Mảng con trỏ hàm: chọn phép biến đổi ★

1. Nêu bài toán

Có ba phép biến đổi trên số nguyên x: bình phương, lập phương và trị tuyệt đối. Dùng mảng con trỏ hàm để chọn phép biến đổi theo mã 1, 2 hoặc 3.

2. Input

choice x, với choice thuộc {1,2,3} và $|x| \leq 10000$.

3. Output

Kết quả phép biến đổi đã chọn.

4. Ví dụ

Input	Output
2 -3	-27

5. Thuật toán giải

Bước 1: Viết ba hàm square, cube, absValue cùng kiểu long long(long long).

Bước 2: Tạo mảng ops chứa địa chỉ ba hàm.

Bước 3: Dùng choice-1 làm chỉ số mảng.

Bước 4: Gọi ops[choice-1](x).

6. Code minh họa

```
#include <iostream>
using namespace std;
```

```

long long square(long long x) { return x * x; }
long long cube(long long x) { return x * x * x; }
long long absValue(long long x) { return x < 0 ? -x : x; }

int main() {
    int choice;
    long long x;
    cin >> choice >> x;

    long long (*ops[3])(long long) = {square, cube, absValue};
    cout << ops[choice - 1](x);
    return 0;
}

```

Bài 28. Con trỏ tới thành viên lớp ★

1. Nêu bài toán

Xây dựng cấu trúc Counter có thuộc tính value và hàm add(). Dùng con trỏ tới dữ liệu thành viên và con trỏ tới hàm thành viên để cộng delta vào value và đọc kết quả.

2. Input

Hai số nguyên initial và delta.

3. Output

Giá trị initial + delta.

4. Ví dụ

Input	Output
10 7	17

5. Thuật toán giải

Bước 1: Tạo Counter c với value = initial.

Bước 2: Khai báo int Counter::* pValue = &Counter::value.

Bước 3: Khai báo con trỏ hàm thành viên pAdd = &Counter::add.

Bước 4: Gọi (c.*pAdd)(delta) và đọc c.*pValue.

6. Code minh họa

```

#include <iostream>
using namespace std;

struct Counter {
    int value;
    void add(int x) { value += x; }
};

int main() {
    int initial, delta;
    cin >> initial >> delta;
    Counter c{initial};

    int Counter::* pValue = &Counter::value;
    void (Counter::*pAdd)(int) = &Counter::add;

    (c.*pAdd)(delta);
    cout << c.*pValue;
}

```

```
return 0;
}
```

Bài 29. Smart pointer unique_ptr cho mảng động

1. Nêu bài toán

Nhập n số nguyên, quản lý mảng động bằng unique_ptr<int[]> thay vì gọi delete[] thủ công. Tìm giá trị nhỏ nhất.

2. Input

n (1 ≤ n ≤ 10000) và n số nguyên.

3. Output

Giá trị nhỏ nhất.

4. Ví dụ

Input	Output
5 4 9 -2 7 1	-2

5. Thuật toán giải

Bước 1: Tạo unique_ptr<int[]> a{new int[n]}.

Bước 2: Nhập dữ liệu qua a[i].

Bước 3: Duyệt để tìm min.

Bước 4: Khi a ra khỏi phạm vi, vùng nhớ mảng được giải phóng tự động.

6. Code minh họa

```
#include <iostream>
#include <memory>
using namespace std;

int main() {
    int n;
    cin >> n;
    unique_ptr<int[]> a{new int[n]};

    for (int i = 0; i < n; ++i) cin >> a[i];
    int mn = a[0];
    for (int i = 1; i < n; ++i)
        if (a[i] < mn) mn = a[i];

    cout << mn;
    return 0;
}
```

Bài 30. shared_ptr và weak_ptr: quan sát vòng đời đối tượng

1. Nêu bài toán

Nhập số nguyên x. Tạo shared_ptr sở hữu x và weak_ptr quan sát cùng đối tượng. In giá trị khi đối tượng còn tồn tại, sau đó reset shared_ptr và kiểm tra weak_ptr đã hết hạn hay chưa.

2. Input

Một số nguyên x.

3. Output

Dòng 1: giá trị x đọc qua weak_ptr khi còn sống. Dòng 2: EXPIRED sau khi chủ sở hữu bị reset.

4. Ví dụ

Input	Output
25	25 EXPIRED

5. Thuật toán giải

Bước 1: Tạo owner = make_shared<int>(x).

Bước 2: Tạo weak_ptr<int> observer = owner.

Bước 3: Dùng observer.lock() để lấy shared_ptr tạm và đọc giá trị.

Bước 4: Gọi owner.reset(); sau đó observer.expired() phải là true.

6. Code minh họa

```
#include <iostream>
#include <memory>
using namespace std;

int main() {
    int x;
    cin >> x;

    shared_ptr<int> owner = make_shared<int>(x);
    weak_ptr<int> observer = owner;

    if (shared_ptr<int> p = observer.lock())
        cout << *p << '\n';

    owner.reset();
    cout << (observer.expired() ? "EXPIRED" : "ALIVE");
    return 0;
}
```

TỔNG KẾT VÀ TÀI LIỆU THAM KHẢO

Bộ 30 bài đi từ raw pointer cơ bản đến các dạng con trỏ thường gặp trong C++11. Mục tiêu của phần raw pointer là giúp người học hiểu mô hình địa chỉ, quan hệ với mảng và cách truyền dữ liệu vào/ra hàm. Khi chuyển sang quản lý quyền sở hữu tài nguyên, bài 29-30 giới thiệu hướng tiếp cận an toàn hơn bằng smart pointer.

Gợi ý tổ chức thực hành

- Bài 1-8 - cú pháp con trỏ, nullptr, const và new/delete.
- Bài 9-19 - con trỏ với mảng, tìm kiếm, biến đổi, pointer arithmetic và mảng động.
- Bài 20-24 - mảng con trỏ, int**, ma trận động, con trỏ tới cấu trúc.
- Bài 25-30 - void*, function pointer, pointer-to-member và smart pointer.
- Bài nâng cao: yêu cầu sinh viên chỉ ra nguy cơ undefined behavior, memory leak, dangling pointer hoặc double delete nếu bỏ các kiểm tra/giải phóng phù hợp.

Tài liệu tham khảo chính

Bjarne Stroustrup. The C++ Programming Language, Fourth Edition. Addison-Wesley, 2013. Các mục tham khảo chính: §7.2-7.4 (Pointers, Arrays), §11.2 (Free Store), §12.2.2 và §12.5 (Array Arguments, Pointers to Functions), §20.6 (Pointers to Members), §34.3.1 (Smart Pointers).

Ghi chú: một số bài cố ý dùng raw pointer và mảng built-in để phục vụ mục tiêu giảng dạy cơ chế ngôn ngữ. Với phần mềm thực tế, nên cân nhắc std::array, std::vector, std::string và smart pointer phù hợp để giảm lỗi quản lý bộ nhớ.